Exercise 4 — REST API and persistence

> Stand-alone version of Exercise 4, for working on your own. The full design brief leaves several architecture questions open on purpose, for a classroom debate. This version settles them into concrete defaults so you can build alone in one session — and lists them at the end for you to argue with afterwards.

What you are building

Take Exercise 2's domain and put a real application around it: Spring Boot, a small REST API, and a database through JPA. No authentication, no security layer. Still specification-driven — the prompt stays small, and the new decisions go into the tables below instead of a long conversation.

This version keeps things as simple as this cohort needs: JPA annotations go **directly on the domain classes** (no separate entity classes, no mapper), and Hibernate **generates** the schema instead of validating against a hand-written one. Neither choice is "the right way" in general — they are simplicity trade-offs for a first pass at persistence, and both are named explicitly below so you know what you gave up.

Architecture

A plain three-layer application, by default. This isn't a ban on hexagonal/ports-and-adapters — if you already have solid, comfortable experience with it, you're welcome to use it. But the default recommendation is the plain version below: the point of this exercise is to feel why that extra complexity earns its cost, not to start there because it's the more fashionable answer. If you're unsure which to pick, ask your instructor before committing — hexagonal is a bigger investment to get right in one sitting than the exercise assumes.

```
HTTP request
  -> PortfolioController   (@RestController)
  -> PortfolioService      (@Service, @Transactional)
  -> PortfolioRepository   (Spring Data JPA)
  -> Portfolio             (domain, now also the @Entity — see below)
  -> H2 database (schema generated by Hibernate)
```

A `@RestControllerAdvice` sits beside the controller, translating domain exceptions into HTTP responses per the error contract below.

The domain underneath

This exercise does not rewrite the domain's behaviour — it wraps it and, this time, annotates it. Bring forward your own working Exercise 2 result (`Portfolio`, `Holding`, `Lot`, the value objects, the exceptions, all 36 tests still passing).

If you don't have one handy, regenerate it fresh first, using Exercise 2's own prompt against `sell-stocks-spec.md` and `domain-class-diagram.puml`. Then, for this exercise, add JPA annotations **directly on those classes**: `@Entity` / `@Table` on `Portfolio`, `Holding` and `Lot`; `@Embeddable` on the value objects (`Money`, `Price`, `ShareQuantity`, `Ticker`); `@Id` on the `*Id` value objects; `@OneToMany` / `@ManyToOne` for the containment relationships; and a package-private no-argument constructor wherever Hibernate needs one to instantiate the class by reflection.

What must not change: every business rule, every public method's behaviour, and all 36 existing tests, unmodified and still green. **What is allowed to change:** the presence of framework annotations, and the minimal structural additions (no-arg constructors, mutable fields where Hibernate needs them) that those annotations require. If you find yourself changing what a method *does* to make JPA happy, stop — that is the interesting result this exercise can produce, and it belongs in a note, not a silent workaround.

The four endpoints

Method	Path	Purpose	Success
`POST`	`/api/portfolios`	Create a portfolio	`201 Created`, portfolio in the body
`GET`	`/api/portfolios/{id}`	Fetch one, with its holdings	`200 OK`
`POST`	`/api/portfolios/{id}/purchases`	Buy shares — sets up the state a sale needs	`200 OK`
`POST`	`/api/portfolios/{id}/sales`	**Sell shares.** The endpoint the whole kata exists for	`200 OK`, proceeds/cost basis/profit in the body

No authentication, no pagination, no filtering. Adding them is a different exercise.

The error contract

Straight from Exercise 2's `error-contract.md` — this exercise is what finally turns it into running, tested behaviour instead of documentation nobody enforces.

Exception	HTTP status	Message pattern
`InvalidQuantityException`	400	`Quantity must be positive: <value>`
`InvalidAmountException`	400	`Price must be positive: <value>`
`InvalidTickerException`	400	`Invalid ticker: <value>` / `Ticker cannot be empty`
`ConflictQuantityException`	409	`Not enough shares to sell. Available: <n>, Requested: <m>`
`HoldingNotFoundException`	404	`Holding not found in portfolio: <ticker>`
`PortfolioNotFoundException`	404	*(new here — a domain-only kata had no repository to raise it)*

Keep the exception messages exact — they become the `detail` field of the error response, and the tests assert on them. One ordering detail carries over from the domain: quantity is validated **before** the holding is looked up, so selling zero shares of a ticker you don't hold is a 400, not a 404.

The schema

Generated by Hibernate from the annotations above, not hand-written — the simpler option for this exercise. Exercise 2's `schema.sql` still exists and describes the same three tables by hand, in case you want to compare what Hibernate produced against a deliberately-designed version:

```
portfolio (portfolio_id PK, owner, balance DECIMAL(19,2), created_at)
holding   (holding_id PK, portfolio_id FK, ticker) — unique (portfolio_id, ticker)
lot       (lot_id PK, holding_id FK, initial_shares, remaining_shares, unit_price,
purchased_at)
          — indexed on (holding_id, purchased_at) for FIFO order
```

Whichever table shape Hibernate lands on, the same limits apply to it. What a generated schema deliberately **cannot** say, and JPA must not silently pretend it can:

- **which lot a sale consumes first** — FIFO is an ordering your code applies, not a database constraint;
- **how proceeds, cost basis, and profit are computed** — a sale returns them; nothing stores or derives them;
- **that every change goes through the portfolio** — a bare `UPDATE lot SET remaining_shares = 0` is legal SQL that nothing here refuses; and
- **that a rejected sale changes nothing** — in the domain that follows from validating before mutating; here it also needs a transaction.

Architecture decisions

Settled as defaults, so you're not debating architecture alone. Each is a legitimate one-line ADR.

Decision	Default	Why
Domain change	Annotations only, no behaviour change	This exercise tests whether a carefully specified domain survives infrastructure with its behaviour unedited — the framework may still leave marks on the class
JPA mapping	Annotations directly on the domain classes, no separate entities or mapper	Simplicity for this cohort — fewer new concepts at once, not a claim that it's the better long-term choice; see "worth discussing afterwards"
Schema authority	Hibernate **generates** the schema (`ddl-auto=create-drop`); `schema.sql` is not used by the app	Also simplicity — one less moving piece to get right in a single session
Transaction boundary	`@Transactional` service methods, load → mutate → save	Makes "a rejected sale changes nothing" provable with a database in the loop
Aggregate loading	The whole `Portfolio`, with its holdings and lots, per request	Simplest option, acceptable at kata scale — the cost is worth discussing afterwards, not fixing now
Concurrency / locking	Out of scope	Worth discussing afterwards; not a requirement here
Identifiers	App-assigned UUID strings from the domain's `*Id` value objects	Keeps the domain in charge of identity, not `@GeneratedValue`

H2 setup

Dependencies: `spring-boot-starter-web`, `spring-boot-starter-data-jpa`, `com.h2database:h2` (runtime scope), `spring-boot-starter-test`.

```
spring.datasource.url=jdbc:h2:mem:portfolio;DB_CLOSE_DELAY=-1
spring.jpa.hibernate.ddl-auto=create-drop
```

H2 is a deliberate choice, not just a convenience: it needs nothing beyond a JAR on the classpath, so it sidesteps any dependency on Docker and Testcontainers, which this training environment may not have cleared for use yet. `ddl-auto=create-drop` rebuilds the schema from your annotations on every run — simplest to work with, and appropriate for a database that only ever holds one test session's data. A real project would revisit both of these: a real datasource (Postgres or MySQL) and a real migration tool (Flyway or Liquibase) instead of letting Hibernate improvise the schema each time.

The suggested prompt

```
Wrap the attached domain model in a Spring Boot REST API with JPA persistence.

Add JPA annotations directly onto the attached domain classes (Portfolio, Holding, Lot, and the
value objects as @Embeddable) rather than creating separate entity classes or a mapper. Keep every
business rule and every public method's behaviour identical — the domain's own 36 tests must still
pass unmodified. The only allowed additions are what JPA requires structurally: annotations,
no-arg constructors, and making a field mutable where Hibernate needs it. If you find yourself
changing what a method does to make JPA happy, stop and tell me why instead of doing it.

Build a plain three-layer application by default — controller, service, repository. (Only reach
for hexagonal/ports-and-adapters if you're already comfortable with it; ask your instructor first
if you're unsure.)

  POST /api/portfolios                create a portfolio
  GET  /api/portfolios/{id}            fetch one, with its holdings
  POST /api/portfolios/{id}/purchases   buy shares
  POST /api/portfolios/{id}/sales       sell shares

Map exceptions to HTTP responses exactly per this table, keeping the exception messages exact:

[ Paste the error contract table here. ]

Let Hibernate generate the schema from these annotations (ddl-auto=create-drop). Don't hand-write
DDL. Use H2 in-memory.

Follow these architecture decisions: [ paste the architecture decisions table here ].

Build the test suite to the scope below — do not re-derive FIFO or acceptance-criterion coverage;
the domain's own 36 tests already own that. This layer only needs to prove wiring, mapping, and
the error contract.
```

```
[ Paste the testing scope table here. ]

Done when the endpoints work end to end against H2, the error contract is enforced with
matching
messages, and the domain's own 36 tests still pass untouched.
```

The testing scope

Real tests, deliberately not exhaustive. About 16 tests total, and explicitly **not** re-verifying FIFO or AC-01…AC-24 — the domain's existing 36 tests already own that; this layer only proves wiring, mapping, and the error contract. There is no separate mapper layer to test here — the `@DataJpaTest` row below is what proves the annotations actually map correctly, since the domain classes are the entities.

Layer	Tool / slice	What it proves	Count
Repository	`@DataJpaTest` (embedded H2)	Save/reload round-trips a `Portfolio` (with its holdings and lots) through the annotated domain classes; cascade delete on holding removal; unknown id returns empty	3
Service	Plain JUnit 5 + Mockito, mocked repository	One test per endpoint's orchestration, plus the one that matters most: a rejected sale never calls `save()`	5
Controller	`@WebMvcTest` + mocked service	One happy path per endpoint, plus one representative 400 and one 404 from the exception handler	6
End to end	`@SpringBootTest(webEnvironment = RANDOM_PORT)` + `TestRestTemplate`, real H2	Full-stack wiring across all four endpoints in sequence, plus one full-stack 409 proving nothing persisted	2

Do not add: Testcontainers or a real Postgres/MySQL instance (H2 avoids the Docker dependency on purpose — see H2 setup above), RestAssured (`MockMvc` and `TestRestTemplate` already ship with `spring-boot-starter-test`), ArchUnit layering enforcement (the architecture is deliberately plain — not worth enforcing structurally at this size), optimistic-locking or retry/concurrency tests, or a performance test for aggregate-loading cost. Those are discussion items below, not test classes.

Worth discussing afterwards

- **Concurrency.** Two sales of the same holding run at once — nothing in the domain or the schema stops both from reading 15 shares and each selling 10. What would optimistic locking with a version column change, and what new acceptance criteria would it need?

- **Aggregate-loading cost.** Loading the whole `Portfolio` on every request is simple. What happens to that choice once a holding has thousands of lots?
- **JPA-on-domain vs. separate entities.** You annotated the domain directly, which is faster to write and couples it to a framework. What would a separate-entities-plus-mapper version have bought you instead, and what would it have cost the first time the domain and the table shape needed to diverge?
- **Generated schema vs. a hand-written one.** Hibernate improvised the schema for you here. Open `schema.sql` from Exercise 2 and compare it to what Hibernate actually created — same tables, same constraints? What breaks the first time this needs a real migration in production, where nobody can afford `create-drop`?

What's next

A conversation gave you working code (Exercise 1). A specification made the behaviour durable (Exercise 2). A skill and a checkpoint made the working method durable (Exercise 3). Here, that same domain's *behaviour* had to survive contact with a database and an HTTP boundary — carrying some new scars in the form of annotations, but not a single changed business rule. Whether it did is the argument the whole kata was building toward.